

PRAKTIKUM SISTEM OPERASI

MODUL 5 Eksplorasi Proses



Oleh:

(Fahmi Hasan Asagaf)

(2311104074)

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

JURNAL/TP

Soal 1

a. Berapa maksimum proses pada Xinu?

Di Xinu, konstanta `NPROC` mendefinisikan jumlah maksimum proses. Nilainya adalah **8** (pada implementasi standar Xinu untuk x86).

Bisa dicek di file `include/process.h`:

C

```
#define NPROC    8
```

b. Berapa maksimal panjang nama proses?

Konstanta `PNMLEN` mendefinisikan panjang nama proses, yaitu **16 karakter**.

C

```
#define PNMLEN    16
```

c. Berapa nilai prioritas awal saat proses dibuat?

Prioritas awal (default) saat proses dibuat adalah **20**.

C

```
#define INITPRIO  20
```

d. Ada berapa total state pada Xinu? Sebutkan!

Total ada **8 state**, yaitu:

Kode	Nama	Keterangan
PR_FREE	free	Slot proses kosong
PR_CURR	curr	Sedang berjalan (current)
PR_READY	ready	Siap dijalankan
PR_RECV	recv	Menunggu pesan
PR_SLEEP	sleep	Sedang tidur/delay
PR_SUSP	susp	Di-suspend
PR_WAIT	wait	Menunggu semaphore
PR_RECTIM	rectim	Menunggu pesan dengan timeout

Di include/process.h:

```
c
#define PR_FREE    0
#define PR_CURR    1
#define PR_READY   2
#define PR_RECV    3
#define PR_SLEEP   4
#define PR_SUSP    5
#define PR_WAIT    6
#define PR_RECTIM  7
```

2. Soal 2 — Komentar 20 Baris Terakhir xsh_ps.c

```
// melakukan perulangan untuk setiap proses

for (i = 0; i < NPROC; i++) {
    prptr = &proctab[i];
    if (prptr->prstate == PR_FREE) { //mengecek apakah slot proses tidak digunakan
        //jika iya, maka proses dilewati(tidak ditampilkan)
        continue;
    }
    printf("%3d %-16s %s %4d %4d 0x%08X 0x%08x %8d %5d %10d\n", //menampilkan informasi seperti pid,nama,state,prioritas,,dll
        i, prptr->prname, pstate[(int)prptr->prstate],
        prptr->prprio, prptr->prparent, prptr->prstkbase,
        prptr->prstkptr, prptr->prstklen,1,prptr->prstklen,0,"-");
}
return 0; // mengembalikan nilai 0 sebagai tanda eksekusi berhasil
```

3.

```

GNU nano 2.2.6                                     File: xsh_ps.c

/* xsh_ps.c - xsh_ps */

#include <xinu.h>
#include <stdio.h>
#include <string.h>

/*-----
 * xsh_ps - shell command to print the process table
 *-----
 */
shellcmd xsh_ps(int nargs, char *args[])
{
    struct procent *prptr;          /* Pointer to process          */
    int32 i;                        /* Index into proctabl        */
    char *pstate[] = {              /* Names for process states   */
        "free ", "curr ", "ready", "recv ", "sleep", "susp ",
        "wait ", "rttime"};

    /* For argument '--help', emit help about the 'ps' command */

    if (nargs == 2 && strncmp(args[1], "--help", 7) == 0) {
        printf("Use: %s\n", args[0]);
        printf("Description:\n");
        printf("\tDisplays information about running processes\n");
        printf("Options:\n");
        printf("\t--help\t display this help and exit\n");
        return 0;
    }

    /* Check for valid number of arguments */

    if (nargs > 1) {
        fprintf(stderr, "%s: too many arguments\n", args[0]);
        fprintf(stderr, "Try '%s --help' for more information\n",
            args[0]);
        return 1;
    }

    /* Print header for items from the process table */

    printf("%3s %-16s %5s %4s %4s %10s %-10s %10s\n",
        "Pid", "Name", "State", "Prio", "Ppid", "Stack Base",
        "Stack Ptr", "Stack Size", "Msg", "Content");

    printf("%3s %-16s %5s %4s %4s %10s %-10s %10s\n",
        "___", "-----", "-----", "-----", "-----", "-----",
        "-----", "-----", "-----", "-----");

    // melakukan perulangan untuk setiap proses

    for (i = 0; i < NPROC; i++) {
        prptr = &proctab[i];
        if (prptr->pstate == PR_FREE) { //mengecek apakah slot proses tidak digunakan
            //jika iya, maka proses dilewati(tidak ditampilkan)
            continue;
        }
        printf("%3d %-16s %s %4d %4d 0x%08X 0x%08x %8d %5d %10d\n", //menampilkan informasi seperti pid,nama,state,prioritas,dll
            i, prptr->prname, pstate[(int)prptr->pstate],
            prptr->prprio, prptr->prparent, prptr->prstkbase,
            prptr->prstkptr, prptr->prstklen, 1, prptr->prstklen, 0, "-");
    }
    return 0; // mengembalikan nilai 0 sebagai tanda eksekusi berhasil
}

```

4.

```
* xsh_uptime - shell to print the time the system has been up
*-----
*/
shellcmd xsh_uptime(int nargs, char *args[])
{
    uint32  days, hrs, mins, secs; /* Days, hours, minutes, and */
                                   /* seconds since system boot */
    uint32  secperday = 86400;     /* Seconds in a day */
    uint32  secperhr  = 3600;     /* Seconds in an hour */
    uint32  secpermin = 60;       /* Seconds in a minute */

    /* For argument '--help', emit help about the 'uptime' command */

    if (nargs == 2 && strcmp(args[1], "--help", 7) == 0) {
        printf("Use: %s\n\n", args[0]);
        printf("Description:\n");
        printf("\tDisplays time since the system booted\n");
        printf("Options:\n");
        printf("\t--help\t display this help and exit\n");
        return 0;
    }

    /* Check for valid number of arguments */

    if (nargs > 1) {
        fprintf(stderr, "%s: too many arguments\n", args[0]);
        fprintf(stderr, "Try '%s --help' for more information\n",
            args[0]);
        return 1;
    }

    secs = clktime; /* Total seconds since boot */

    /* Subtract number of whole days */

    days = secs/secperday;
    secs -= days*secperday;

    /* Subtract number of hours */

    hrs = secs/secperhr;
    secs -= hrs*secperhr;

    /* Subtract number of minutes */

    mins = secs/secpermin;
```